Git 入門課：學生練習講義

這份講義把課堂操作整理成可獨立閱讀、列印的版本。你可以搭配投影片使用，也可以依序閱讀；Sxx 是投影片對照 ID，方便回頭查找對應畫面。

五個練習使用同一個小專案，請先把教材倉庫中的 `exercises/grade-lab` 複製到教材倉庫外，當作獨立副本。每個練習都在該副本中進行；完成後再回到投影片或本講義核對概念。

1. 開始之前

課前準備

- 安裝 Git[1]，版本 2.28 以上。Windows 使用 Git Bash；macOS / Linux 使用終端機。
- 準備可保存純文字的編輯器。使用 VS Code 且已安裝 `code` 命令時，可設定如下；已有其他可用編輯器就不必更換：

```
git config --global core.editor "code --wait"
```

- 設定 commit 作者資訊時，把範例改成自己的姓名與電子郵件；這不是 GitHub 登入。也可使用 GitHub 設定頁提供的 noreply 電子郵件。

```
git --version
git config --global user.name "Alex Chen"
git config --global user.email "alex@example.com"
```

- 準備 GitHub 帳號，依 GitHub 官方認證指南[2] 使用 Git Credential Manager 或 GitHub CLI。CLI 需先安裝，再執行 `gh auth login`，選 HTTPS 並同意設定 Git 認證。已有可用的 SSH 設定也可以使用，但本講義使用 HTTPS 網址。
- 課前請用自己的測試倉庫確認 `push` 成功。不要把 Token 放進 remote 網址、檔案或截圖。
- Python 是選用的檢查工具，不是完成 Git 練習的必要條件。依環境使用 `python`、`python3` 或 `py`。

[1] `https://git-scm.com/downloads`
[2] `https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git`

Git 與 GitHub 的分工

Git 是安裝在本機的版本控制工具；GitHub 是存放 Git 倉庫的遠端服務。沒有 GitHub 仍可 `init`、`add`、`commit`，只是 commit 只留在本機。GitHub 讓他人取得、審閱與備份遠端倉庫。

準備獨立的練習目錄（S05）

用檔案管理器把本教材的 `exercises/grade-lab` **整個複製**到個人主目錄，改名為 `git-grade-lab`。目標必須是新目錄，不可覆蓋既有專案。已有同名目錄時，改用新名稱，並相應調整後續本機路徑。

打開 Git Bash / 終端機，進入副本：

```
cd "$HOME/git-grade-lab"
pwd
```

應看到以下兩個起始檔案，沒有 `.git`：

```
git-grade-lab/
  README.md
  grades.py
```

起始 `README.md`：

```
# Grade calculator

Calculate the average of a list of scores.
```

起始 `grades.py`：

```
def average(scores):
    return sum(scores) / len(scores)


print("Average:", average([70, 80, 90]))
```

可以向 AI 求助，但課堂上請親手輸入每一條命令。

每次編輯後先在編輯器保存。不要輸入 shell 的 `$` 提示符號。除非另外說明，命令都在目前的練習倉庫中執行。

2. 保存與檢查版本

工作目錄與暫存區

工作目錄是正在編輯的檔案；暫存區記錄下一次 commit 要包含的內容；本機倉庫保存已完成的 commit。`git add` 把工作目錄的變更放入暫存區，`git commit` 把暫存區寫成新 commit。

`git status` 顯示目前 branch、已暫存與未暫存變更、未追蹤檔案。`git diff` 比較工作目錄與暫存區；`git diff --staged` 比較暫存區與最後一次 commit（`HEAD`）。因此同一檔案可同時有這兩種差異。

工作目錄 $\xrightarrow{\texttt{add}}$ 暫存區 $\xrightarrow{\texttt{commit}}$ 本機倉庫 $\xrightarrow{\texttt{push}}$ 遠端倉庫

練習 1：分開保存兩次修改（S07–S11）

建立第一個版本

```
git init -b main
git status
git add README.md grades.py
git status
git commit -m "Add grade calculator"
git status
```

預期目前位於 `main`，已有第一個 commit，工作目錄乾淨。

先預測，再 commit

1. 在 README 末尾加入 `Input: a list of scores.`，保存。
2. 執行 `git add README.md`。
3. 再追加 `Output: the average score.`，保存，**先不要 add**。
4. 執行下面三個命令，並預測下一個 commit 會包含哪一行。

```
git status
git diff
git diff --staged
```

先只 commit 輸入說明，再把輸出說明保存成另一個 commit。

完成條件：至少有三個 commit；Input 與 Output 分別 commit；工作目錄乾淨；能說明兩種 `diff` 各自比較的對象。

查看參考答案（第 13 頁）

小檢查：忽略不需要分享的檔案（S12）

建立 `.gitignore`，內容如下。註解必須獨立成行。

```
# Local credentials
.env
# Generated Python files
__pycache__/
*.pyc
```

建立 `.env`，只放假資料 `DEMO_ONLY=example`。確認 `.env` 沒有出現在一般 `status` 中，再保存共用規則：

```
git status
git add .gitignore
git commit -m "Ignore local and generated files"
git status
```

這些規則對未追蹤檔案生效；已追蹤的檔案不會自動停止追蹤。忽略也不會清除歷史中的內容。

查歷史與選復原方式

`git log --oneline` 看 commit 順序，`git show HEAD` 看目前 commit 的內容。復原前先判斷變更在哪裡：尚未暫存、已暫存但未 commit，或已經 commit。三種情況用不同命令，不能互相取代。

取消暫存用 `git restore --staged README.md`，只把暫存區回復到 `HEAD`，工作目錄的修改仍在。丟棄未暫存修改用 `git restore README.md`，會用暫存區版本覆蓋工作目錄；這會失去修改，先確認不要了再執行。兩者是選項，不要依序全部執行。

已 commit 的變更若已分享，通常用 `git revert` 新增 revert commit 來抵銷，保留歷史。它不會刪掉舊 commit；若只是本機尚未分享的錯誤，才在理解後考慮其他整理方式。

練習 2：撤銷一次錯誤，保留歷史（S13–S15）

先確定工作目錄乾淨，把 README 的**第一行**改為 `# Broken calculator`，其餘行保留。

```
git add README.md
git commit -m "Change heading by mistake"
```

```
git log --oneline
git show HEAD
```

確認 HEAD 是剛才的錯誤 commit，再執行：

```
git revert --no-edit HEAD
git log --oneline
git status
```

完成條件：標題恢復為 `# Grade calculator`，輸入/輸出說明仍在，工作目錄乾淨；歷史中同時有錯誤 commit 與抵銷 commit。`revert` 會新增 revert commit，不會刪掉舊 commit。

以下兩種常見情境是**選項，不要接著依序執行**：

想做的事	命令	工作內容
取消 README 的暫存	`git restore --staged README.md`	保留工作目錄修改
丟棄 README 未暫存的修改	`git restore README.md`	用暫存區版本覆蓋；先確定修改不要了

3. branch 與合併

branch、HEAD 與合併方向

branch 名稱是指向某個 commit 的可移動標籤；建立 branch 不會複製整個目錄。HEAD 通常指向目前所在 branch，commit 後該 branch 前進，HEAD 也跟著指向新 commit。

`git switch -c handle-empty` 從目前 commit 建立 branch 並切過去；往後 commit 記在 `handle-empty`。`git merge handle-empty` 要在 `main` 上執行，因為合併目標是目前 branch，`handle-empty` 是來源。

本例 `main` 自 branch 建立後沒有新 commit，因此這次合併可以快轉：`main` 直接移動到 `handle-empty` 的 commit，不產生 merge commit。

練習 3：在 branch 上開發，再合併回 `main`（S16–S19）

以下操作只需執行一次。若已建立 `handle-empty` branch，直接核對完成條件，不要重複建立同名 branch。

```
git status
```

```
git switch -c handle-empty
```

把 grades.py 的函式改為以下版本，保留原本的 print：

```
def average(scores):
    if not scores:
        return 0
    return sum(scores) / len(scores)
```

今天的專案約定：空清單的平均值回傳 0。

```
git add grades.py
git commit -m "Handle empty score lists"
git switch main
```

先開啟 grades.py：main 應還沒有空清單判斷。再合併：

```
git merge handle-empty
git branch
git log --oneline --graph --all
git status
```

完成條件：目前在 main，檔案已有空清單判斷，工作目錄乾淨。main 與 handle-empty 此時指向同一 commit，來源 branch 名稱仍存在。

可選驗證；依環境替換 python 命令：

```
python -c "from grades import average; assert average([70,80,90]) == 80;
↪  assert average([]) == 0"
```

內容衝突與解決步驟

合併時若兩邊修改同一段內容，Git 無法判斷保留哪一版，會停下來標示衝突。這是需要人工判斷的情況：先讀兩邊的用意，再寫出同時保留雙方資訊的版本，而不是只選一邊。

衝突標記中的 HEAD 是目前 branch（main）的版本，另一段是來源 branch（improve-title）的版本。編輯檔案時移除 <<<<<<<、=======、>>>>>>>，留下整理後的內容。

解決後執行 git add README.md，表示衝突已處理並放入暫存區；再執行 git commit 完成 merge commit。add 不會自動判斷內容好壞，commit 才建立合併結果。

練習 4：整合兩邊的標題修改（S20–S23）

開始前位於 `main`，工作目錄乾淨。若已建立 `improve-title` 並完成兩邊 commit，直接核對完成條件，不必重跑本節。

```
git switch -c improve-title
```

把 README 第一行改為 `# Team grade calculator`，其他內容保留，再執行：

```
git add README.md
git commit -m "Describe team calculator"
git switch main
```

把 `main` 的 README 第一行改為 `# Class grade calculator`，其他內容保留：

```
git add README.md
git commit -m "Describe class calculator"
git merge improve-title
```

預期 `merge` 會停下來，並提示 README 有衝突。執行 `git status`，打開 README，找到：

```
<<<<<<< HEAD
# Class grade calculator
=======
# Team grade calculator
>>>>>>> improve-title
```

將整段替換成 `# Team grade calculator for class`，保留其餘說明。確認沒有留下衝突標記，再執行：

```
git add README.md
git commit -m "Combine team and class descriptions"
git log --oneline --graph --all
git status
```

完成條件：`main` 的標題保留 Team 與 class 的意思；輸入/輸出說明仍在；沒有衝突標記；有 merge commit 且工作目錄乾淨。

若想取消**尚未完成**的合併，可以執行 `git merge --abort`。本練習從乾淨工作目錄開始；不要假設它能恢復任何未保存的內容。

4. 分享與協作

遠端、推送與 PR

`origin` 是 clone 時預設使用的遠端別名，也是常見習慣，不是固定必須使用的名稱；`main` 是 branch 名稱；`origin/main` 是本機記住的遠端 `main` 狀態。`commit` 只更新本機倉庫；`push` 把本機 commit 送到遠端並更新遠端 branch；`fetch` 會取得本機缺少的 commit 等物件，並更新 `origin/main` 等遠端追蹤資訊，不改工作目錄。

PR 是 GitHub 上的審閱與合併請求，不是 Git 命令。推送來源 branch 後開 PR；之後在同一個來源 branch commit 並 `push`，原 PR 會更新。PR 合併後，本機 `main` 還要用 `pull` 取得遠端結果。

遠端與本機各有新 commit 時，推送可能因非快轉被拒絕。這通常只是歷史分歧：先 `fetch`，再在乾淨工作目錄合併 `origin/main`。若兩邊沒有改到同一段內容，整合通常可自動完成；若 Git 無法判斷保留哪一版，會停下來標示衝突，需照練習 4 人工解決。

練習 5：推送、審閱並更新同一個 PR（S24–S29）

推送本機倉庫

確認目前在 `main`。到 GitHub 建立自己擁有的空倉庫 `git-grade-lab`，**不新增 README、`.gitignore` 或授權檔**。只使用不含敏感資料的課堂專案；公開倉庫方便同學查看與留言，私有倉庫需事先安排存取權限。

把 `YOUR-USER` 換成自己的 GitHub 帳號；若倉庫另有名稱，也修改網址中的 `git-grade-lab`。

```
git remote add origin https://github.com/YOUR-USER/git-grade-lab.git
git push -u origin main
```

到 GitHub 確認可以看到 README、`grades.py` 與 commit 紀錄，且不應包含 `.env`。

建立文件 branch 與 PR

```
git switch -c docs-usage
```

在 README 追加用法，例如：

````
## Usage

Run the calculator with Python:

~~~bash
python grades.py
~~~

Expected output: Average: 80.0
````

若使用 `python3` 或 `py`，請相應修改說明。

```
git diff
git add README.md
git commit -m "Explain how to run the calculator"
git push -u origin docs-usage
```

在 GitHub 新增 PR，base 選 `main`，compare 選 `docs-usage`。描述可使用：

```
目的：讓第一次取得這個專案的同學知道如何執行。
變更：新增執行命令與預期輸出。
檢查：核對命令與檔名；若環境有 Python，實際執行並記錄結果。
```

雙人審閱與更新

1. 各自提出自己的 PR，交換連結。審閱者檢查用法、檔名與預期結果，提出一個具體建議；不需要對方的 `push` 權限。
2. 作者依意見修改本機 README，並維持在 `docs-usage` branch。
3. 保存後 commit 並推送，**原 PR 會更新**：

   ```
   git add README.md
   git commit -m "Clarify usage after review"
   git push
   ```

4. 在原 PR 回應意見，並重新檢查 Files changed。
5. 由有權限的人在 GitHub 合併 PR；本課可選 Create a merge commit。
6. 在本機更新 `main`：

```
git switch main
git pull --ff-only
git status
```

完成條件：PR 有目的、檢查說明與審閱紀錄；已依意見更新同一個 PR；PR 已合併；本機 `main` 有最終用法，工作目錄乾淨。

補充：clone 與遠端更新（S26–S27，可選）

這個小節是選用補充，可在完成五個練習後閱讀；以下在新目錄取得既有倉庫，不再執行 `init`：

```
git clone https://github.com/YOUR-USER/git-grade-lab.git git-grade-copy
cd git-grade-copy
git status
```

`origin/main` 是本機記住的遠端狀態。`fetch` 會更新它，不會直接改動工作檔案。如果 `main` 兩邊各有新 commit，先確定工作目錄乾淨，再處理非快轉推送被拒絕的情況：

```
git fetch origin
git merge --no-edit origin/main
```

若有衝突，照練習 4 編輯、`add`、`commit`；無衝突就直接完成整合，再執行 `git push`。認證失敗或沒有寫入權限是另外的問題，不適用這個處理方式。完成補充後回到原本的練習目錄；第二份 `clone` 只用於觀察遠端更新，不用來完成 PR 練習。

5. 課後想一想（S30）

請先嘗試作答並記下想法：

1. add 後又改檔案，下次 commit 會保存哪一版？
2. commit 成功後，GitHub 一定看得到嗎？
3. 將功能 branch 合併進 main 前，要切到哪裡？
4. 編輯完有衝突的檔案後，還需要做什麼？
5. 如何更新同一個 PR？

查看參考答案（第 13 頁）

6. 排查補充

依現象排查

先確認目前目錄、branch 與 `git status`，再依現象檢查。下表只列先檢查什麼；處理時避免使用會覆寫他人歷史或丟棄未保存內容的命令。

現象	先檢查
命令輸出停在分頁器	畫面底部是否顯示 `:`；按 `q` 離開。必要時用 `git --no-pager log --oneline` 取得單次輸出。
同一檔案同時有已暫存與未暫存修改	分別看 `git diff --staged` 與 `git diff`；確認下一次 commit 要包含哪一版，再決定 add 或保留工作目錄修改。
建立 branch 時提示同名 branch 已存在	用 `git branch` 確認；切到既有 branch，或改用新名稱，不要重複建立。
推送被拒絕且提示非快轉	先 `git fetch`，檢查 `origin/main` 與本機 `main` 的差異；在工作目錄乾淨時合併，再推送。不要用強制推送。
認證失敗或沒有寫入權限	確認 remote URL 的協定（HTTPS 或 SSH）與認證方式相符；講義範例多用 HTTPS，但既有可用的 SSH 也可使用。確認帳號對該遠端倉庫有權限；不要把 Token 放進網址。

需要幫忙時

先說明目前目錄、branch、想做的事與錯誤訊息，並附上 `git status` 與相關 `diff`。可查文件，或問同學、助教、AI；執行建議前先理解它會改動哪個狀態，不要直接貼出憑證或私人內容。

Pro Git 繁體中文	`https://git-scm.com/book/zh-tw/v2`
git diff	`https://git-scm.com/docs/git-diff`
git restore	`https://git-scm.com/docs/git-restore`
git revert	`https://git-scm.com/docs/git-revert`
gitignore	`https://git-scm.com/docs/gitignore`
git pull	`https://git-scm.com/docs/git-pull`
GitHub Pull Requests	`https://docs.github.com/en/pull-requests`

7. 參考答案

練習 1：分開保存兩次修改（S07–S11）

完成預測後再讀以下操作。

`git diff` 顯示未暫存的 Output；`git diff --staged` 顯示已暫存的 Input。同一個檔案可以同時有這兩種變更。

```
git commit -m "Document calculator input"
git diff
git add README.md
git commit -m "Document calculator output"
git log --oneline
git status
```

第一次 `commit` 後，Output 仍在工作目錄，並未包含在那次 commit 中。

返回題目（第 3 頁）

課後想一想（S30）

以下逐題對照。

1. 暫存區中最後一次 `add` 當時的內容。
2. 不一定，還需 `push` 到遠端。
3. `main`；`merge` 的目標是目前所在 branch。
4. 檢查內容、執行 `git add`，再執行 `git commit` 完成合併。
5. 在 PR 的來源 branch 修改、commit 並 `push`，原 PR 會更新。

返回題目（第 11 頁）